Using the /proc filesystem

Introduction

- This lab introduces the /proc virtual filesystem in Linux.
- This lab expands on the pthread & dining philosopher lab
- Build and tag the starter code:

```
cd eecs678-procfs-lab/; make; ctags -R
```

The ps Command

- Linux distributions usually come with several programs for monitoring system activity.
- The *ps* displays information about processes on the system.
- The -*ux* flags lists processes started by the current user.
- The -*aux* includes processes belonging to all users.
  - Run *ps -ux* here

Process Statistics

- %CPU: The CPU utilization of the process.
  - This is the ratio of time spent on the CPU over the real time since it was started
- %MEM: ratio of the processe's resident memory as a share of physical memory.
- VSZ and RSS: virtual and resident memory, in KB.
- START and TIME: start time and total CPU time.
- COMMAND: the command that started the process.

top

- top is interactive version of *ps*.
- top refreshes its statistics periodically, every 3 seconds by default in the example.
  - Run *top -u <username>* with your own username.
- Choose different sort columns and display additional statistics.
- Type h for commands and q to quit.
- See *man top* for details.

The /proc Filesystem

- *ps* and *top* are user level processes.
- However the information requires a view of the entire system. *top* needs this information to be updated frequently. Thus, many OS level structures need to expose information to user programs
- How do systems provide this type of information to user-level processes?
  - Event logging/tracing requires disk space and overhead to create logs
  - Direct access to kernel memory requires user level programmers to have knowledge of kernel data structures
- Linux exposes process and system information through the hierarchical /proc filesystem.

A Virtual File System

- In Linux everything is a file, including processes
- /proc uses the file model to present dynamically changing aspects of the process state
- Process entries appear and disappear as processes live and die
  - Many files in /proc are created at boot and are destroyed at shutdown.
  - These files do not occupy ordinary physical storage.
- We can use familiar file interfaces and shell commands to interact with /proc

Listing Files

- *ls* /proc lists system files and numbered process directories.
- *ls -l* shows permissions, ownership, size, and timestamps.
- Lets look at an example on the next slide

Listing Files

```
$ ls -l /proc/cpuinfo

-r--r--r-- 1 root root 0 2026-11-02 04:57 /proc/cpuinfo
```

- This information means /proc/cpuinfo:
  - Is a regular file as indicated by – if it was a directory the first symbol would be a d
  - Has read permissions for the owner group and user but no write or execute
  - Contains 1 hard link (itself). This would be more interesting if /proc/cpuinfo were a directory
  - Is owned by root and is part of the group root
  - Has a size of 0 on the disk again this is because /proc is virtual
  - Has a time stamp of November 2 at 04:57. The time the file was last updated
  - Is located at /proc/cpuinfo

Viewing Top-Level Files

- Lets read the cpuinfo file using
  - *cat /proc/cpuinfo*
- This might give you something like

```
processor  : 7

vendor_id  : GenuineIntel
cpu family : 6
model      : 15
model name : Intel(R) Xeon(R) CPU X5355 @ 2.66GHz
stepping   : 7
cpu MHz    : 2659.987
cache size : 4096 KB
```

Viewing Top-Level Files

- Processor IDs begin at zero, so processor 7 is the eighth processor.
- The example identifies an Intel Xeon X5355 rated at 2.66 GHz, with a 4096 KB cache.
- The observed running speed at boot time in the example is 2659.987 MHz. Almost exactly the 2.66 GHz its rated for

Other Important Files

- There are several other files in /proc that might be interesting to explore on your own
- We recommend /proc/meminfo, /proc/net/, and /proc/uptime as files you may find interesting
- Also check out the *man proc* for an indepth explanation of these files
- Commands such as *free*, *netstat*, and *uptime* present related information in a readable form.
- For this lab focuses on the numbered directories that hold process state information

Process Directories

- Each numbered /proc directory corresponds to a process (PID) of a running process
- Within each directory there are files and subdirectories that expose data the kernel maintains for that process.
- Find your shell PID with *ps ux*, then list /proc/<PID>/.
- Here we see several files, too many to get into in a single lab. So we'll zoom into a couple to show you one powerful way to use /proc

Using /proc in Your Programs

- Recall that the dining philosophers alternate between thinking and eating.
- Previously, each thread updated progress variables and the main thread monitored them for deadlock.
- Linux /proc statistics offer another way to observe thread progress. Although it is less portable
- The starter code for this lab implements the dining philosopher's problem, but the main threads check_for_deadlock() function has been left empty for you to implement using /proc

The Stat File

- Each /proc/<PID> directory contains a file called ‘stat’ which contains process statistics in a form programs can parse.
- We provided *./procstat <PID>* which prints a human readable version of stat.
- For this lab we will focus on two entries in this file. The running time in user mode and the running time in system mode
  - Note the unit of time is the Linux timer tick called jiffies
- We can gather both times and check if they’ve changed since the last check. If they have, we can infer the program is still running and if they have not we can infer the program has reached a state of deadlock

Opening Threads' Stat Files

- Each process directory in /proc has a link called self/ which allows a process to refer to its /proc entry without knowing its own PID.
- Each multithreaded process's /proc directory contains a directory called task/ which includes subdirectories for each thread
- /proc/self/task numbers its directories by the thread ID of each thread in the process group just like how /proc uses PIDs

Opening Threads' Stat Files

- We can get each thread ID using the *gettid()* call provided because getting this to work is a bit subtle.
  - *gettid()* system call is not portable and thus we have to wrap it in this syscall() function. We've done this for you at the top of each philosopher thread
- Now we can form a filename for each of the stat files of the threads we with to monitor by using the sprintf() library call wit the string "/proc/self/task/%d/stat" and providing the i'th diner's thread id by passing diners[i].tid as the final agrument

Scanning to the Data We Want

- Our approach is to scan this file until we reach the data we want
- Looking at man proc we see that user and system times are the 14th and 15th fields on the stat file
- You will need to use *fopen()*to open a file stream to each thread's stat file. Consider using the provided variables in check_for_deadlock().
- Next, you can use *fscanf()* calls to scan over the first 13 fields of data then read and store the 14th and 15th fields
- Check compiler warnings for missing or malformed fscanf() arguments.

fscanf()

```
int fscanf(FILE *stream, const char *format, ...);
```

- fscanf() works like scanf(), but reads from a file stream passed in as its first parameter.
- sscanf does the same thing while reading strings
- By using an optional character * in the format specifier, fscanf() reads input as directed by he conversion specification but discards the input

```
fscanf(filep, "%*s");
/ Read and discard one string field read from filep
```

Checking for Deadlock

- Once you have the times stored away in local variables you can check for deadlock.
- A global symbol “DEADLOCK” controls how the program compiles
  - A value of zero avoids deadlock
  - A value other than zero causes the program to deadlock
- ACTIVE_DURATION controls how long eating and thinking take
- The check_for_deadlock() routine is incomplete, but the structure assumes deadlock has occurred and looks for evidence of progress
- This is where you should think about how the user and system time values can represent progress. As a hint you need to use the user and system time arrays to check for deadlock. The progress arrays should only be updated for printing later
- Finally when you are through be sure to close the open file stream with fclose()

Testing the Solution

- When you are through, you should ensure that your solution works in situations where deadlock occurs and when it does not occur.
- Test the default DEADLOCK = 1 and the asymmetric DEADLOCK = 0 version.
- Note that larger ACTIVE_DURATION values make deadlock less likely, and smaller values make it more likely

Final Output

- Print the sampled user and system times for all five philosophers.
- The source example shows the following pairs of timing values:

```
Philosopher   P0        P1        P2        P3        P4
User time     245/245   238/238   212/212   220/220   217/217
System time   0/0       0/0       0/0       0/0       0/0

User time     205/450   204/442   180/392   183/403   179/396
System time   1/1       1/1       1/1       0/0       1/1
```